

PSEL Seminar

Continuous control with deep reinforcement learning

Timothy P. Lillicrap✱ , Jonathan J. Hunt✱ , Alexander Pritzel, Nicolas Heess, Tom Erez,
Yuval Tassa, David Silver & Daan Wierstra

Seohyeon Kim

**Department of Industrial and Management Engineering
Incheon National University**

■ Backgrounds

- Deep Reinforcement Learning
 - Approximation in value space
 - Approximation in policy space
 - Actor - Critic



■ Motivation

■ DDPG: Deep Deterministic Policy Gradient

- Deterministic policies to handle continuous action spaces
- DQN technique
- Actor-Critic, off-policy
- Exploration vs. Exploitation
- DDPG Algorithm

■ Results and Discussion

- DDPG is able to learn good policies

■ DDPG: Deep Deterministic Policy Gradient

✓ Model-Free, Off-policy, Actor-Critic

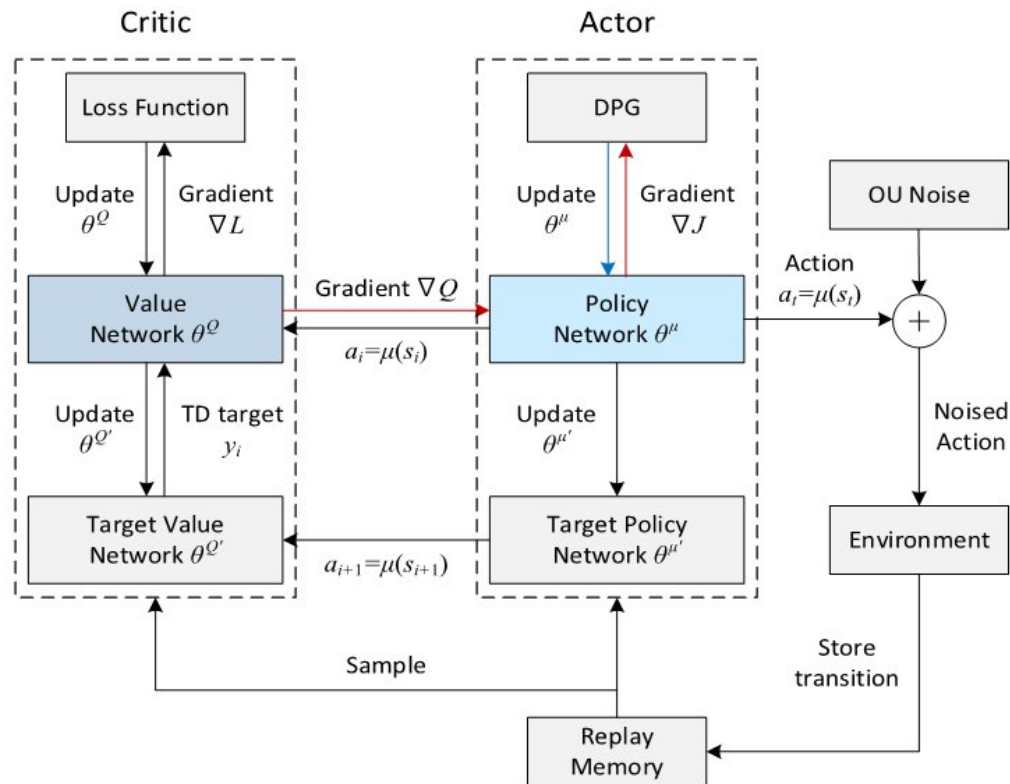


Fig. Overview of DDPG

■ Reinforcement Learning (강화학습)

- Learn in an interactive environment by trial and error
- Agent chooses an action at each state to **maximize the total cumulative rewards**
- ✓ *Markov decision process (MDP): a stochastic decision-making process*

⇒ Model-based: known MDP (transition probability) vs Model-free: unknown MDP

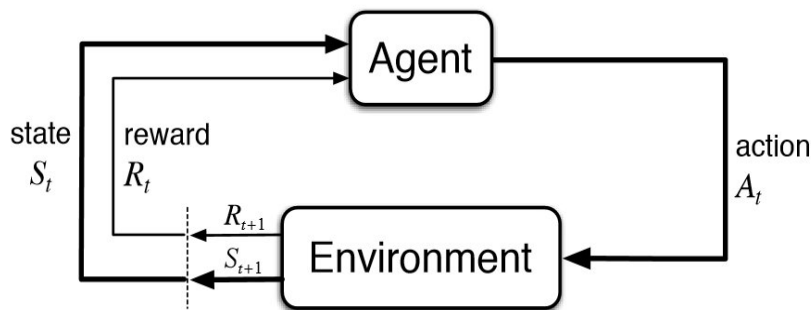


Fig. Architecture of RL

$$A_t \sim \pi(\cdot | S_t), \quad \forall t > 0$$

$$\pi(a|s) = \mathbb{P}[A_t = a | S_t = s]$$

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

■ Reinforcement Learning (강화학습)

- Learn in an interactive environment by trial and error
- Agent chooses an action at each state to **maximize the total cumulative rewards**
- ✓ **Bellman equation : expected return after taking an action a_t in state s_t and thereafter following policy π**

$$Q^{\pi}(s_t, a_t) = \mathbb{E}_{r_t, s_{t+1} \sim E} [r(s_t, a_t) + \gamma \mathbb{E}_{a_{t+1} \sim \pi} [Q^{\pi}(s_{t+1}, a_{t+1})]]$$

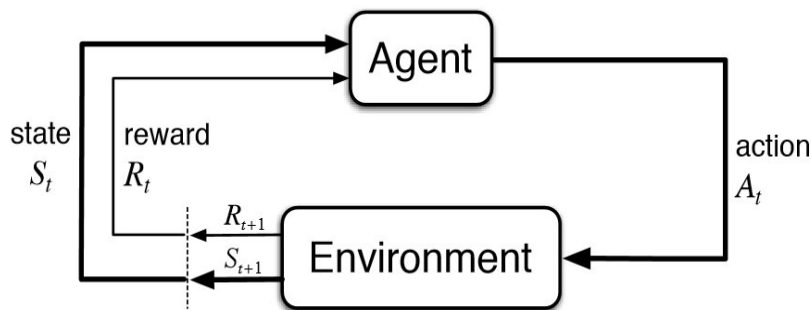


Fig. Architecture of RL

$$\begin{aligned} v_{\pi}(s) &= \mathbb{E}_{\pi}[G_t | S_t = s] \\ &= \mathbb{E}_{\pi}[R_{t+1} + \gamma v_{\pi}(S_{t+1}) | S_t = s] \end{aligned}$$

expected return starting from state s when following policy π

$$\begin{aligned} q_{\pi}(s, a) &= \mathbb{E}_{\pi}[G_t | S_t = s, A_t = a] \\ &= \mathbb{E}_{\pi}[R_{t+1} + \gamma q_{\pi}(S_{t+1}, A_{t+1}) | S_t = s, A_t = a] \end{aligned}$$

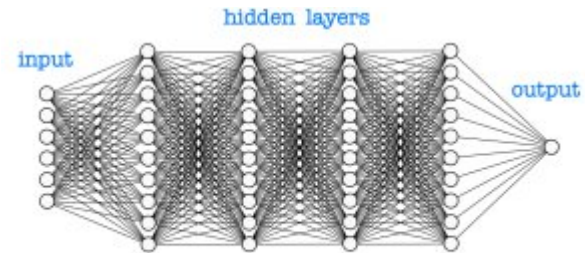
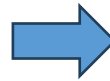
expected return starting from state s , taking action a ,
and following policy π

▪ Deep Reinforcement Learning (심층강화학습)

- 강화학습(RL)과 심층 신경망(DNN)의 결합
 - DNN: enables automatic feature engineering of high dimensional data
- è 매우 큰 state space, action space를 갖는 환경에서 신경망을 이용하여 학습하여 주어진 state에서 중요한 feature만을 추출
- è 실제 환경에서의 복잡한 의사결정 문제에 효과적 (e.g. 자율주행 차량, 로봇 제어)

	↑	↓	←	→
1	0.49	0.44	0.45	0.41
2	0.40	0.40	0.43	0.42
3	0.48	0.41	0.40	0.29
⋮	⋮	⋮	⋮	⋮
9	0.77	0.57	0.66	0.85

$$Q(s, a)$$



$$Q(s, a; \theta)$$

■ Deep Reinforcement Learning (심층강화학습)

- 강화학습(RL)과 심층 신경망(DNN)의 결합
 - DNN: enables automatic feature engineering of high dimensional data
 - è 매우 큰 state space, action space를 갖는 환경에서 신경망을 이용하여 학습하여 주어진 state에서 중요한 feature만을 추출
 - è 실제 환경에서의 복잡한 의사결정 문제에 효과적 (e.g. 자율주행 차량, 로봇 제어)

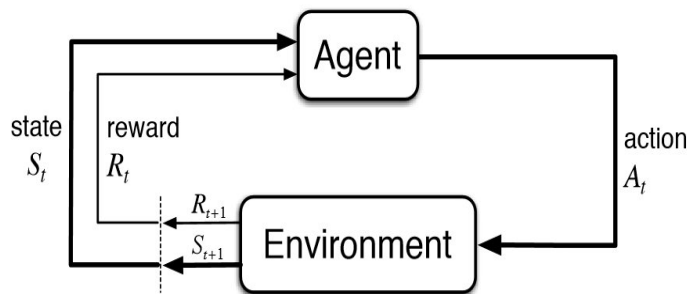


Fig. Architecture of RL

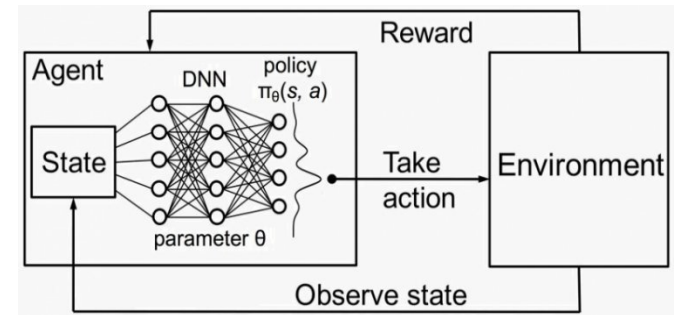
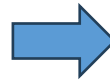


Fig. Architecture of DRL

■ Deep Reinforcement Learning - Approximation in value space

- ✓ Bellman equation : expected return after taking an action a_t in state s_t and thereafter following policy π

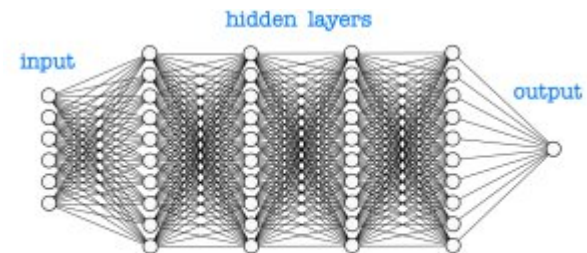
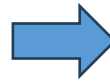
$$Q^\pi(s_t, a_t) = \mathbb{E}_{r_t, s_{t+1} \sim E} [r(s_t, a_t) + \gamma \mathbb{E}_{a_{t+1} \sim \pi} [Q^\pi(s_{t+1}, a_{t+1})]]$$

Action value function

- Value space를 비선형 함수로 근사
- 파라미터 θ 를 학습하여 optimal value function에 근사

	↑	↓	←	→
1	0.49	0.44	0.45	0.41
2	0.40	0.40	0.43	0.42
3	0.48	0.41	0.40	0.29
⋮	⋮	⋮	⋮	⋮
9	0.77	0.57	0.66	0.85

$Q(s, a)$



$Q(s, a; \theta)$

▪ Deep Reinforcement Learning - Approximation in value space

- ✓ DQN: Deep Q-Network
- ✓ DQN enables human level control

Q - learning $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t)]$

$Q(s, a; \theta)$?

Naïve Deep Q - learning $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t)]$

$$L(\theta) = \underbrace{[r_{t+1} + \gamma \max_a Q(s_{t+1}, a; \theta)]}_{\text{target}} - \underbrace{Q(s_t, a_t; \theta)}_{\text{Estimate value}}]^2$$

▪ Deep Reinforcement Learning - Approximation in value space

- ✓ DQN: Deep Q-Network
- ✓ DQN enables human level control

Q - learning $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t)]$

$Q(s, a; \theta)$?

Naïve Deep Q - learning $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t)]$

$$L(\theta) = \underbrace{[r_{t+1} + \gamma \max_a Q(s_{t+1}, a; \theta)]}_{\text{target}} - \underbrace{Q(s_t, a_t; \theta)}_{\text{Estimate value}}]^2$$

è Temporal correlations between samples

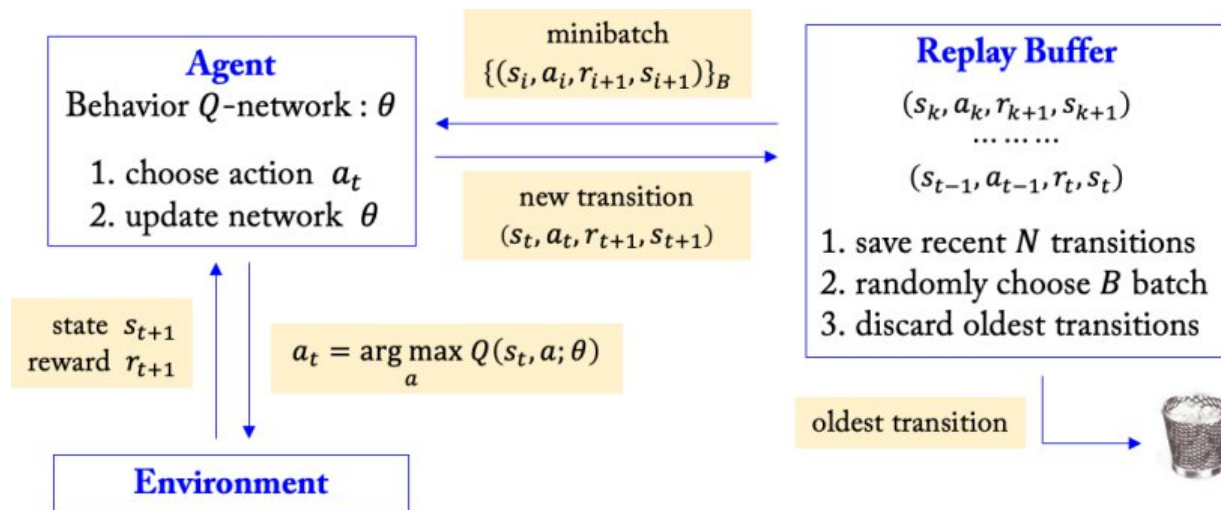
è Non – stationary targets

▪ Deep Reinforcement Learning - Approximation in value space

- ✓ DQN: Deep Q-Network
- ✓ DQN enables human level control

è Temporal correlations between samples

è Experience Replay (Replay Buffer)



▪ Deep Reinforcement Learning - Approximation in value space

- ✓ DQN: Deep Q-Network
- ✓ DQN enables human level control

è **Non – stationary targets**

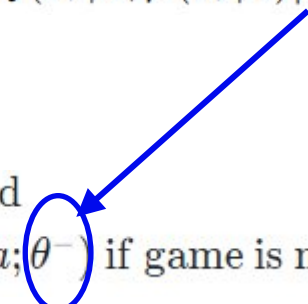
è **Target Network**

$$\theta^{Q'} \leftarrow \theta^Q$$

- Q_{θ^-} : target network
- Q_{θ} : current value

$$L(\theta^Q) = \mathbb{E}_{s_t \sim \rho^\beta, a_t \sim \beta, r_t \sim E} \left[(Q(s_t, a_t | \theta^Q) - y_t)^2 \right]$$

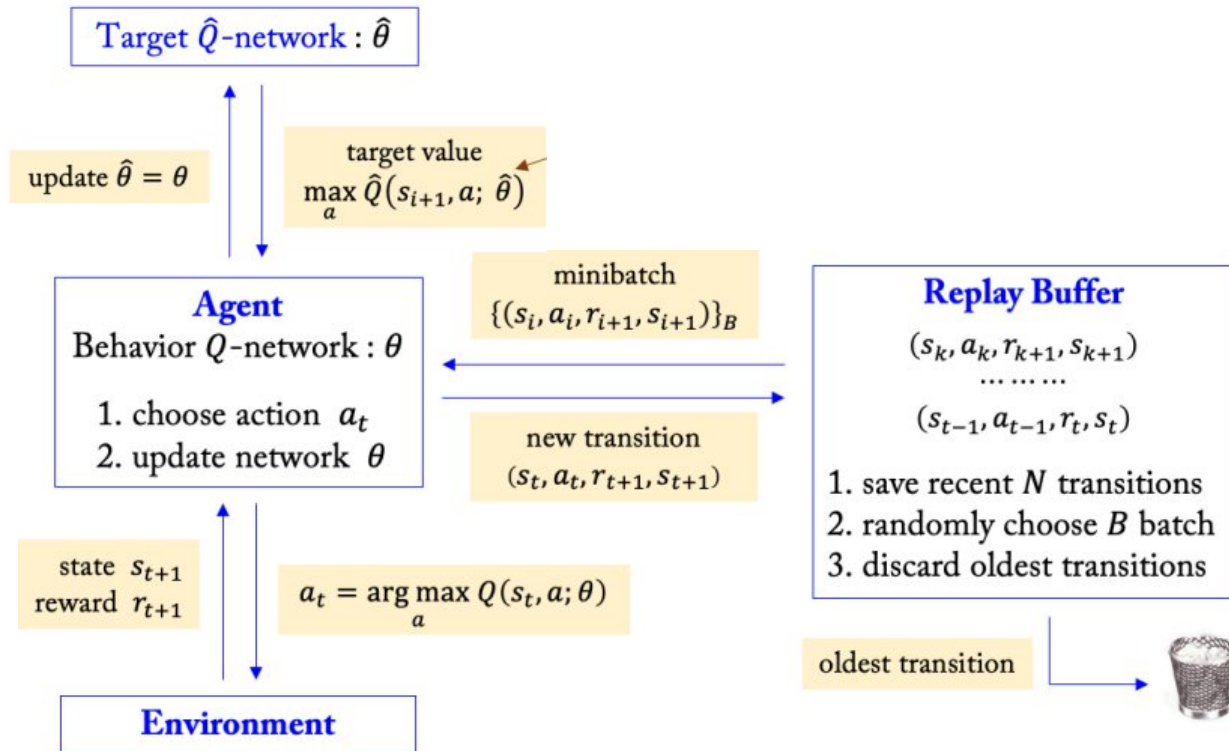
$$y_t = r(s_t, a_t) + \gamma Q(s_{t+1}, \mu(s_{t+1}) | \theta^Q).$$

$$y = \begin{cases} r & \text{if game is finished} \\ r + \gamma \max_a Q(s_{t+1}, a; \theta^-) & \text{if game is not finished} \end{cases}$$


Target 값이 변동하지 않음

■ Deep Reinforcement Learning - Approximation in value space

- ✓ DQN: Deep Q-Network
- ✓ DQN enables human level control



■ Deep Reinforcement Learning - Approximation in policy space

- ✓ Bellman equation : expected return after taking an action a_t in state s_t and thereafter following policy π

$$Q^\pi(s_t, a_t) = \mathbb{E}_{r_t, s_{t+1} \sim E} [r(s_t, a_t) + \gamma \mathbb{E}_{a_{t+1} \sim \pi} [Q^\pi(s_{t+1}, a_{t+1})]]$$

Action value function

$$\mu : \mathcal{S} \leftarrow \mathcal{A}$$

$$Q^\mu(s_t, a_t) = \mathbb{E}_{r_t, s_{t+1} \sim E} [r(s_t, a_t) + \gamma Q^\mu(s_{t+1}, \mu(s_{t+1}))]$$

Deterministic target policy

- 파라미터 θ 를 학습하여 optimal policy에 근사
- Action을 parameterize, continuous action을 나타내는 장점
- Goal: maximize expected return of J

$$\nabla_{\theta^\mu} J \approx \mathbb{E}_{s_t \sim \rho^\beta} [\nabla_{\theta^\mu} Q(s, a | \theta^Q) |_{s=s_t, a=\mu(s_t | \theta^\mu)}]$$

■ Deep Reinforcement Learning - Approximation in policy space

- ✓ Bellman equation : expected return after taking an action a_t in state s_t and thereafter following policy π

$$Q^\pi(s_t, a_t) = \mathbb{E}_{r_t, s_{t+1} \sim E} [r(s_t, a_t) + \gamma \mathbb{E}_{a_{t+1} \sim \pi} [Q^\pi(s_{t+1}, a_{t+1})]]$$

Action value function

$$\mu : \mathcal{S} \leftarrow \mathcal{A}$$

$$Q^\mu(s_t, a_t) = \mathbb{E}_{r_t, s_{t+1} \sim E} [r(s_t, a_t) + \gamma Q^\mu(s_{t+1}, \mu(s_{t+1}))]$$

Deterministic target policy



Stochastic policy
 $\pi_\theta(a|s) = P[a|s; \theta]$

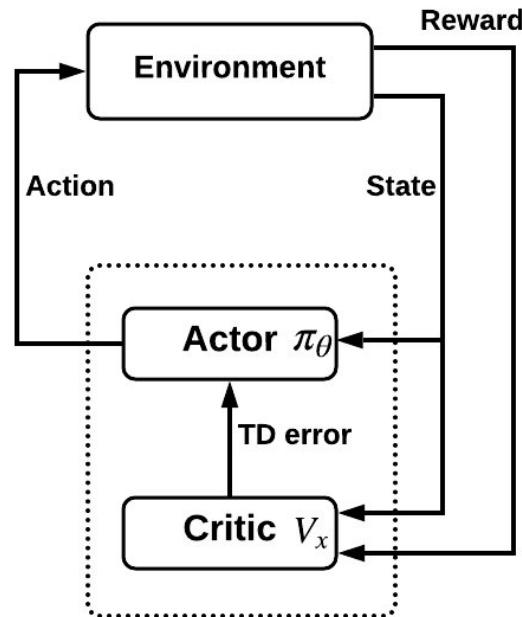


Deterministic policy
 $a = \mu_\theta(s)$

▪ Deep Reinforcement Learning – Actor – Critic

è Critic network updates parameter φ for value function $V(s; \varphi)$ **or** $Q(s, a; \varphi)$

è Actor network updates parameter θ for policy $\pi_{\theta}(a | s)$ using Policy Gradient $\nabla_{\theta} J(\theta)$



▪ Deep Deterministic Policy Gradient

- ✓ DQN: processing high-dimensional observation spaces, it still handles only low-dimensional discrete action spaces
- ✓ Most real world problem have continuous and high dimensional action spaces



Fig. Playing Atari with Deep Reinforcement Learning

Continuous action space

Deep Deterministic Policy Gradient

- ✓ Model-Free, Off-policy, Actor-Critic algorithm

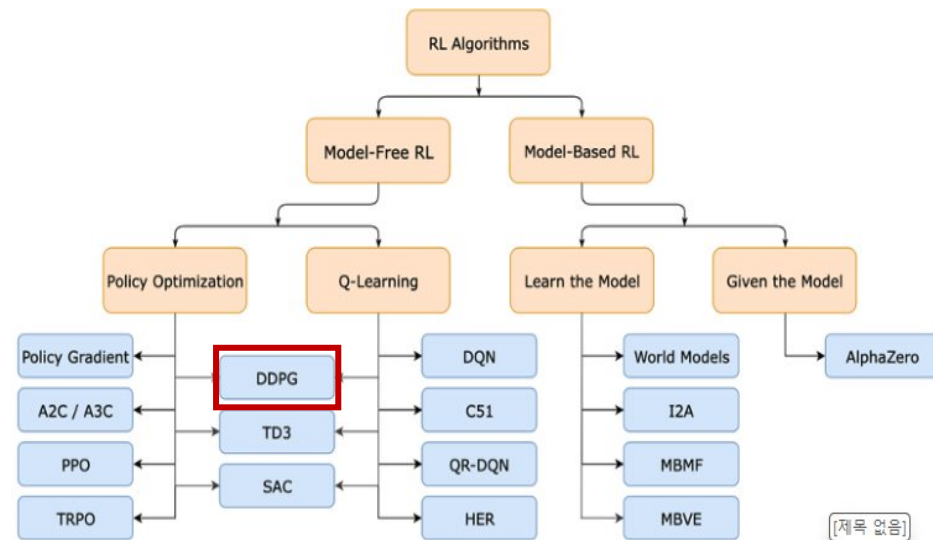
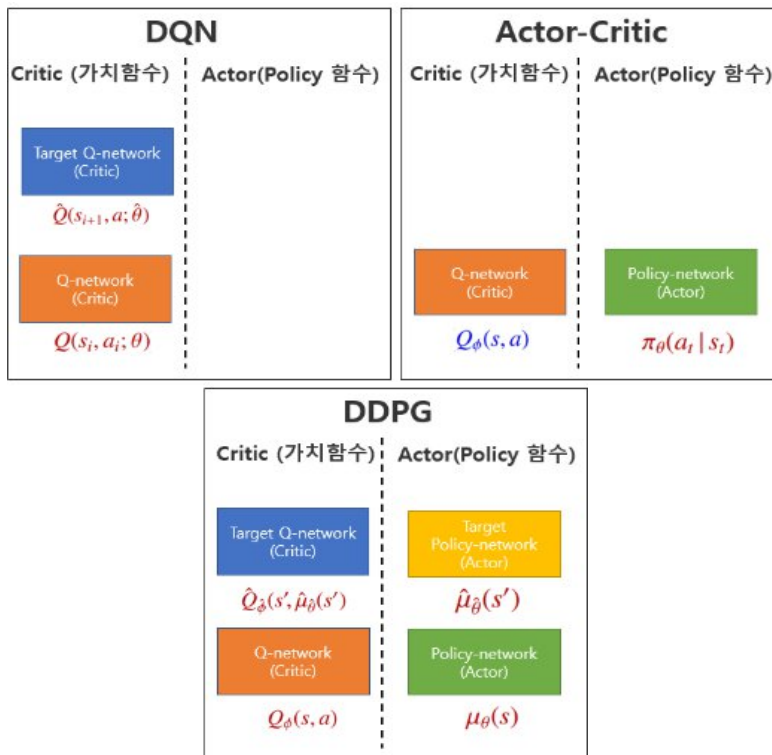


Fig. Architecture of DRL

▪ Deterministic policies to handle continuous action spaces

- ✓ based on Deterministic Policy Gradient (DPG)
 - **(Stochastic)Policy Gradient Theorem** vs. Deterministic Policy Gradient Theorem
 - State에 따른 policy를 학습하기 위해 고안됨
 - Stochastic policy probability function $\pi(a|s)$

Max

$$J(\theta) = \mathbb{E}_{\pi}[R]$$

Object function

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{s,a \sim \pi} [\nabla_{\theta} \log \pi_{\theta}(a|s) \cdot Q^{\pi}(s, a)]$$

Policy update

$$\Delta \theta = Q^{\pi}(s,a) \nabla_{\theta} \log \pi_{\theta}(a|s)$$

▪ Deterministic policies to handle continuous action spaces

- ✓ based on Deterministic Policy Gradient (DPG)
 - (Stochastic)Policy Gradient Theorem vs. **Deterministic Policy Gradient Theorem**
 - Deterministic policy $a = \mu(s)$ → 상대적으로 적은 sample을 필요로 함
 - DPG learns deterministic policy as an actor on continuous action spaces and an action-value function as a critic

Object function

$$\nabla_{\theta} J(\theta) = \mathbb{E}_s [\nabla_{\theta} \mu_{\theta}(s) \cdot \nabla_a Q^{\mu}(s, a)|_{a=\mu_{\theta}(s)}]$$

Policy update

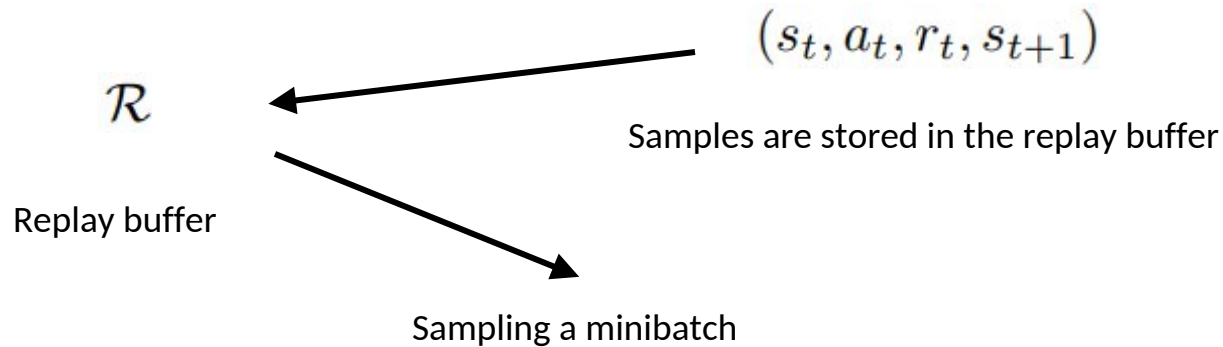
$$\Delta \theta = \underbrace{\nabla_a Q^{\mu}(s, a)|_{a=\mu(s)}}_{\text{Critic}} \underbrace{\nabla_{\theta} \mu_{\theta}(s)}_{\text{Actor}}$$

Critic

Actor

▪ DQN technique

- ✓ Experience replay(replay buffer) / Target network



Experience replay

$$L = \frac{1}{N} \sum_i (y_i - Q(s_i, a_i | \theta^Q))^2$$

$$y_i = r_i + \gamma Q'(s_{i+1}, \mu'(s_{i+1} | \theta^{\mu'}) | \theta^{Q'})$$

Update network

$$\theta^{Q'} \leftarrow \tau \theta^Q + (1 - \tau) \theta^{Q'}$$

Update target network(soft update)

Target network

▪ Actor-Critic, off-policy

- ✓ Actor : Q-function $Q(s,a)$
- ✓ Critic: **deterministic policy**

	Loss	Target network update
Actor	$\nabla_{\theta^\mu} J \approx \frac{1}{N} \sum_i \nabla_a Q(s, a \theta^Q) _{s=s_i, a=\mu(s_i)} \nabla_{\theta^\mu} \mu(s \theta^\mu) _{s_i}$	$\theta^{\mu'} \leftarrow \tau \theta^\mu + (1 - \tau) \theta^{\mu'}$
Critic	$L = \frac{1}{N} \sum_i (y_i - Q(s_i, a_i \theta^Q))^2$	$\theta^{Q'} \leftarrow \tau \theta^Q + (1 - \tau) \theta^{Q'}$

■ Exploration vs. Exploitation

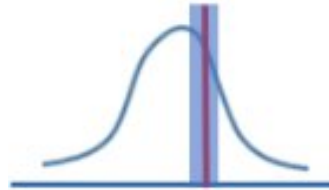
- ✓ To make DDPG policies explore better, we add noise to their actions at training time.
 - Continuous action space: 약간의 noise를 더해 exploration을 유도

Actor policy

$$\mu'(s_t) = \mu(s_t|\theta_t^\mu) + \mathcal{N}$$

Ornstein-Uhlenbeck Process

시간적으로 연관된 확률변수 생성



▪ DDPG Algorithm

Algorithm 1 DDPG algorithm

Randomly initialize critic network $Q(s, a|\theta^Q)$ and actor $\mu(s|\theta^\mu)$ with weights θ^Q and θ^μ .

Initialize target network Q' and μ' with weights $\theta^{Q'} \leftarrow \theta^Q$, $\theta^{\mu'} \leftarrow \theta^\mu$

Initialize replay buffer R

for episode = 1, M **do**

 Initialize a random process $\mathcal{N}$ for action exploration

 Receive initial observation state s_1

for $t = 1, T$ **do**

 Select action $a_t = \mu(s_t|\theta^\mu) + \mathcal{N}_t$ according to the current policy and exploration noise

 Execute action a_t and observe reward r_t and observe new state s_{t+1}

 Store transition (s_t, a_t, r_t, s_{t+1}) in R

 Sample a random minibatch of N transitions (s_i, a_i, r_i, s_{i+1}) from R

 Set $y_i = r_i + \gamma Q'(s_{i+1}, \mu'(s_{i+1}|\theta^{\mu'})|\theta^{Q'})$

 Update critic by minimizing the loss: $L = \frac{1}{N} \sum_i (y_i - Q(s_i, a_i|\theta^Q))^2$

 Update the actor policy using the sampled policy gradient:

$$\nabla_{\theta^\mu} J \approx \frac{1}{N} \sum_i \nabla_a Q(s, a|\theta^Q)|_{s=s_i, a=\mu(s_i)} \nabla_{\theta^\mu} \mu(s|\theta^\mu)|_{s_i}$$

 Update the target networks:

$$\theta^{Q'} \leftarrow \tau \theta^Q + (1 - \tau) \theta^{Q'} \quad \text{Actor (Value Network)}$$

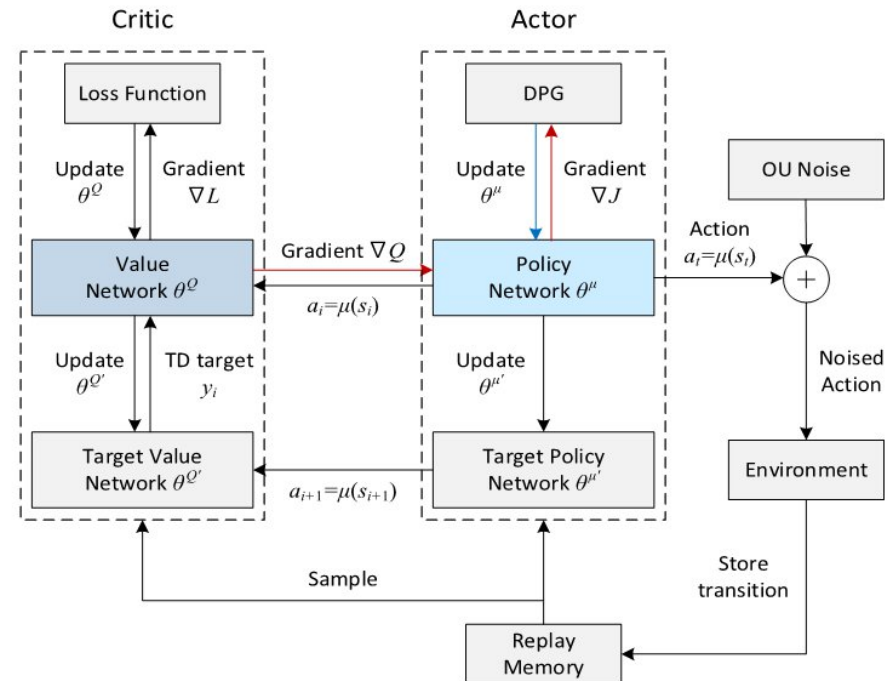
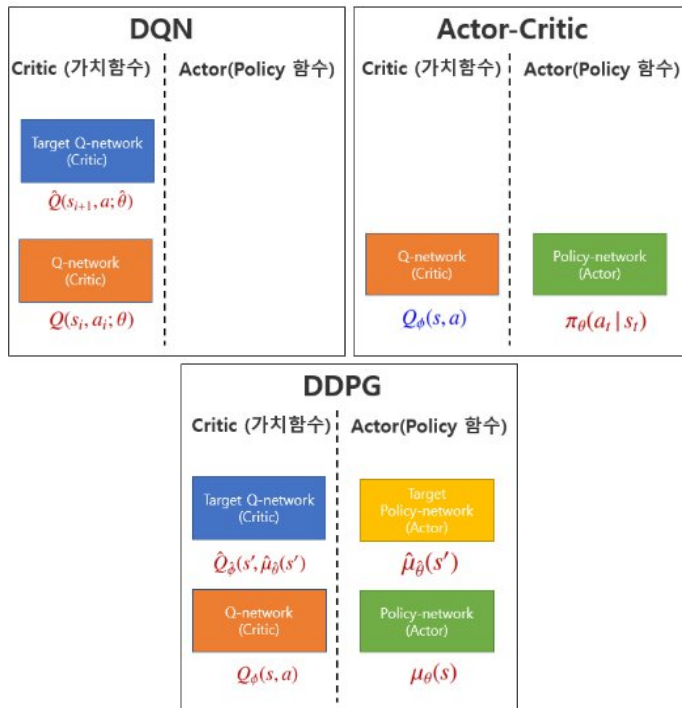
$$\theta^{\mu'} \leftarrow \tau \theta^\mu + (1 - \tau) \theta^{\mu'} \quad \text{Critic (Deterministic Policy Network)}$$

end for

end for

Deep Deterministic Policy Gradient

- ✓ Model-Free, Off-policy, Actor-Critic algorithm
- ✓ Learning **deterministic policies** to handle **continuous action spaces** by adapting DQN architecture such as **experience replay** and **target network**



- **DDPG is able to learn good policies**
 - Performance results of a sample of environments

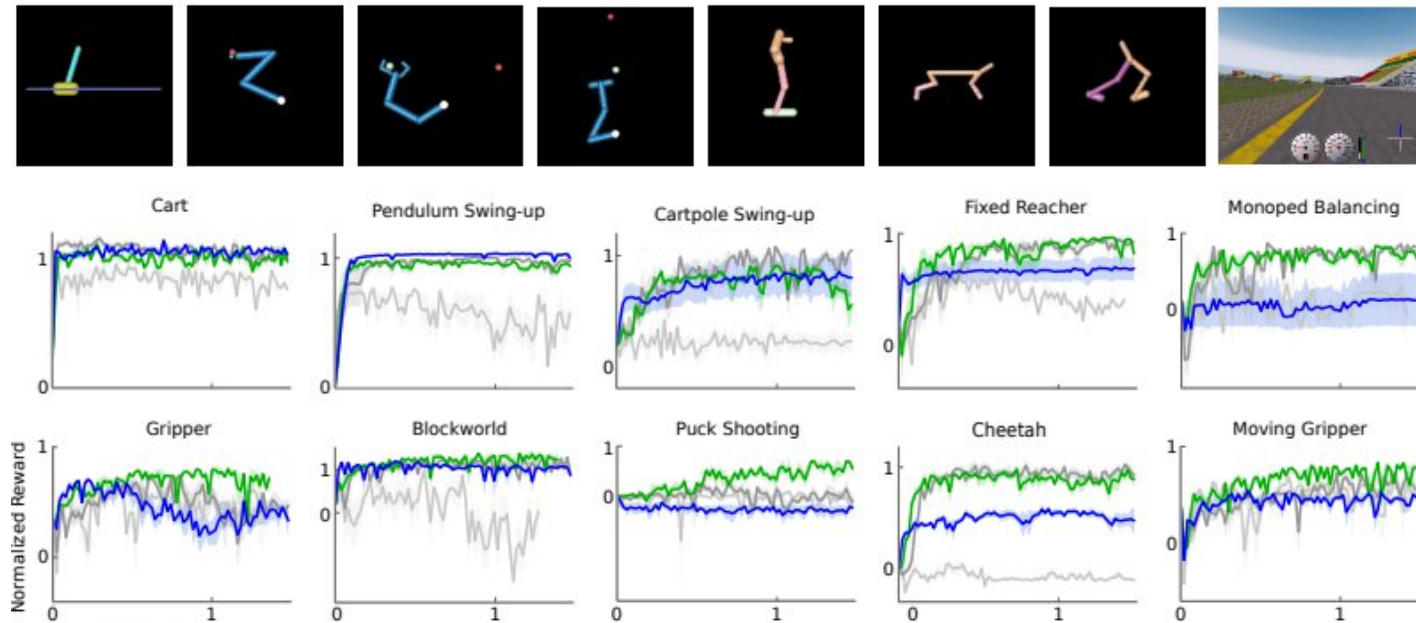


Fig. Performance curves for a selection of domains using variants of DPG

- DPG with batch normalization
- DPG with target network
- **DPG with target net + batch normalization**
- **DPG with target networks from pixel-only inputs**

- **DDPG is able to learn good policies**

- merits and limitation

Merits

- Strong in difficult problems

#continuous space
#deep network

- robust

#replay buffer
#target network

limitation

- Requires a large number of training episodes

#model-free

PSEL Seminar

Thank you

Continuous control with deep reinforcement learning

Seohyeon Kim
Department of Industrial and Management
Engineering
Incheon National University